Lab 10-12: Implementation of RISC-V Processor in Verilog

Lab Objectives:

- Understand RISC-V Architecture, including instruction formats, opcode decoding, and register operations.
- Understand the working of RISC V Architecture.
- Learn single-cycle processor design methodology.
- Implement instruction fetch, decode, execute, and write-back stages in Verilog.
- Develop skills in test bench development and simulation for verification.

Introduction:

RISC-V is an open-source instruction set architecture (ISA) that is revolutionizing the way we design and implement computer processors. Developed at the University of California, Berkeley, RISC-V is a modern, modular, and extensible ISA that is designed to be simple, efficient, and scalable.

History of RISC-V:

RISC-V was first developed in 2010 by Yunsup Lee, Andrew Waterman, and David Patterson at the University of California, Berkeley. The initial version, RISC-V v1, was a 32-bit ISA that was designed to be a simple and efficient alternative to existing ISAs. Since then, RISC-V has evolved to include 64-bit and 128-bit versions, as well as various extensions and modifications.

Key Features of RISC-V:

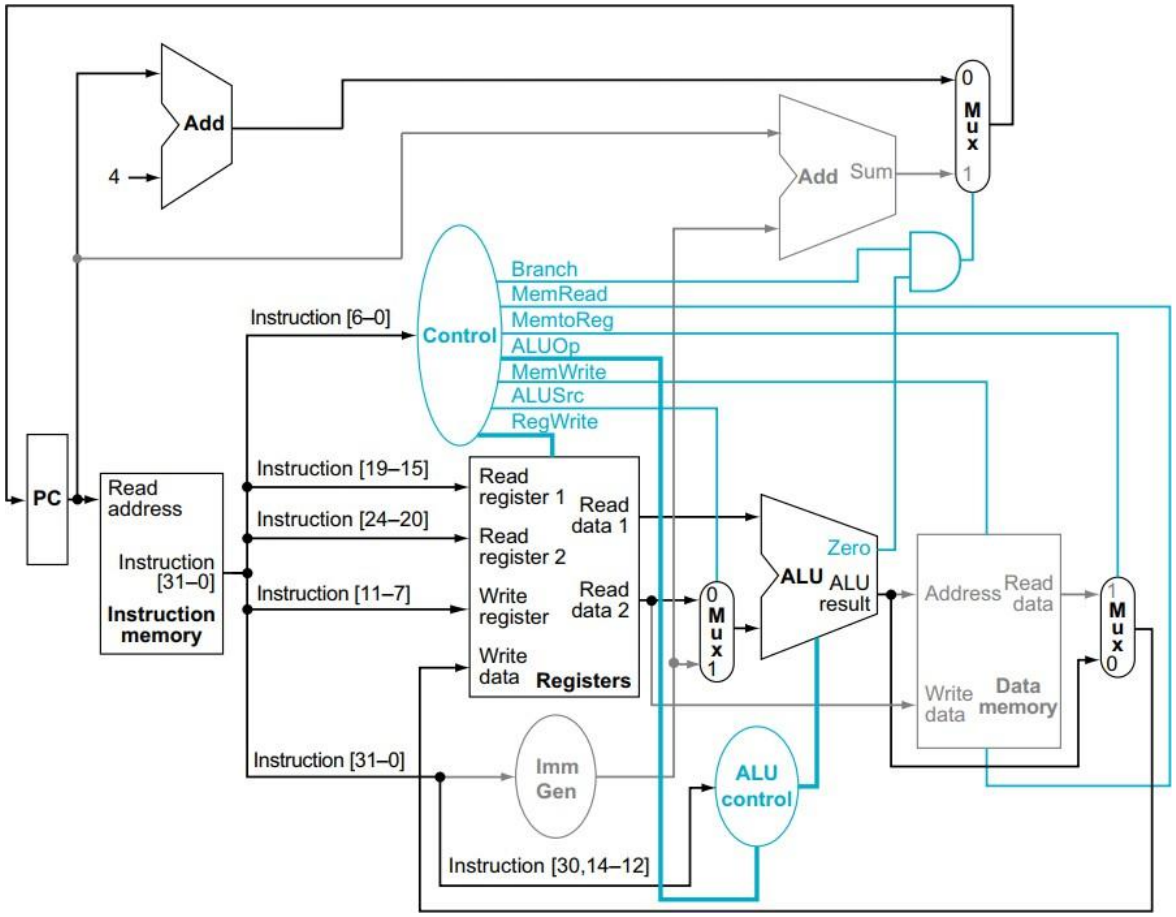
RISC-V has several key features that make it an attractive choice for processor design:

- Simple and Modular: RISC-V has a simple and modular design, with a small number of instructions and a regular instruction format.
- Extensible: RISC-V is designed to be extensible, with a large number of reserved instruction codes and a modular design that allows for easy addition of new instructions and features.
- Open-Source: RISC-V is an open-source ISA, which means that anyone can use, modify, and distribute it without restriction.
- Free: RISC-V is free to use, with no licensing fees or royalties.
- High-Performance: RISC-V is designed to be high-performance, with a focus on efficiency and scalability.

Advantages of RISC-V:

RISC-V has several advantages over other ISAs:

- Customizability: RISC-V's modular design and open-source nature make it easy to customize and modify for specific applications.
- Portability: RISC-V is designed to be portable, with a focus on software compatibility across different implementations.
- Security: RISC-V has a number of security features, including hardware support for secure boot and secure execution.



The simple datapath for the core RISC-V architecture combines the elements required by different instruction classes

Main Components:

All the main components in the above figure are already created in previous labs.
Here's a brief introduction of each component.

Program Counter (PC):

- The Program Counter (PC) is a special-purpose register that holds the address of the next instruction to be fetched from memory.
- It is automatically incremented after each instruction fetch operation.
- In a RISC-V processor, the PC is essential for maintaining the instruction execution sequence and determining the memory address of the next instruction to fetch.

Adder:

- An adder is a digital circuit responsible for performing addition operations on binary numbers.
- In a RISC-V processor, adders are commonly used for various purposes such as incrementing the PC to fetch the next instruction, computing memory addresses for load and store operations, and performing arithmetic calculations within the ALU.
- They can be simple ripple-carry adders or more complex carry-lookahead or carry-save adders, depending on the performance requirements of the processor.

Multiplexers (MUXes):

- Multiplexers, often abbreviated as MUXes, are digital circuits that select one of several input signals and route it to the output based on control signals.
- In a RISC-V processor, MUXes are used to select between multiple sources of data or control signals, depending on the current instruction being executed or the processor's state.
- They play a crucial role in datapath design by enabling dynamic routing of data and control signals to various functional units such as the ALU, register file, and memory units.

Arithmetic Logic Unit (ALU):

- The Arithmetic Logic Unit (ALU) is a digital circuit responsible for performing arithmetic and logical operations on binary data.
- In a RISC-V processor, the ALU executes operations specified by instructions fetched from memory, such as addition, subtraction, AND, OR, and XOR.
- It operates on data stored in registers or memory and produces results based on the instruction operands.
- The ALU's efficiency and speed significantly impact the overall performance of the processor, making it a crucial component in datapath design.

Register File:

- The register file stores temporary data and operands for arithmetic and logic operations.
- It typically consists of a set of registers accessible by the processor's execution units.
- RISC-V processors often have a small number of general-purpose registers for frequently used data.
- These registers are directly accessible by the instruction set architecture (ISA) and play a crucial role in instruction execution.
- The register file facilitates fast access to operands, reducing memory access overhead and enhancing performance.

Instruction Memory:

- The instruction memory is a storage unit that holds the program instructions to be executed by the processor.
- It stores the binary representation of instructions fetched from the program stored in main memory or cache.
- In a RISC-V processor, the instruction memory is typically implemented using static random-access memory (SRAM) or read-only memory (ROM) technologies.
- Instructions are fetched from the instruction memory sequentially based on the address provided by the program counter (PC).
- The instruction memory is a critical component in the instruction fetch stage of the processor's pipeline, providing instructions to the instruction decode stage for further processing.
- Its speed and efficiency directly impact the overall performance of the processor, making it essential for efficient instruction execution.

Immediate Generation Unit:

- The immediate generation unit generates immediate values used as operands in instructions.
- It extracts immediate fields from instruction words and sign-extends them to the required width.
- In RISC-V processors, the immediate generation unit plays a crucial role in processing instructions with immediate operands.
- It ensures that immediate values are correctly formatted and ready for use in arithmetic, logical, or branching operations.
- The immediate generation unit enhances instruction execution efficiency by providing immediate values directly to the ALU or other functional units.

Control Unit:

- Coordinates the flow of data and activities within the processor.
- Generates control signals based on instruction opcodes and processor state.
- Manages instruction fetch, decode, execute, and writeback stages.
- Activates specific functional units such as ALU and memory units as required.
- Ensures proper instruction execution according to the RISC-V ISA.
- Vital for maintaining overall processor operation and efficiency.

ALU Control Unit:

- Determines the ALU operation based on the instruction opcode.
- Interprets instruction opcodes to generate appropriate control signals.
- Specifies ALU operations for addition, subtraction, logical AND, OR, XOR, etc.
- Ensures the ALU operates correctly according to the executed instruction.
- Coordinates ALU activities with the instruction execution process.
- Essential for enhancing the processor's ability to efficiently handle diverse computational tasks.

Data Memory:

- The data memory unit is responsible for storing and retrieving data from memory.
- It includes the data cache and memory management logic necessary for accessing memory locations.
- In RISC-V processors, the data memory unit interacts with the memory hierarchy to access main memory or cache levels efficiently.
- It handles load and store operations, transferring data between the processor and memory.
- The data memory unit's performance influences memory access latency and overall system throughput.

Lab Task:

Design and implement a single-cycle RISC-V processor in Verilog, using the examples and code from past labs as a guide.

Here are the steps you can follow to implement the RISC-V Processor.

Step 1: Design the Instruction Memory

- Define the instruction memory size
- Design the instruction memory module
- Write Verilog code for instruction memory

Step 2: Design the Data Memory

- Define the data memory size
- Design the data memory module
- Write Verilog code for data memory

Step 3: Design the Program Counter

- Define the program counter width (e.g. 32 bits)
- Design the program counter module
- Write Verilog code for program counter

Step 4: Design the Adders

- Define the adder width (e.g. 32 bits)
- Design the adder module
- Write Verilog code for adder

Step 5: Design the Decoder

- Define the instruction formats (R-type, I-type, S-type, SB-type)
- Design the decoder module
- Write Verilog code for decoder

Step 6: Design the Immediate Generation

- Define the immediate width (e.g. 12 bits)
- Design the immediate generation module
- Write Verilog code for immediate generation

Step 7: Design the ALU Control Unit

- Define the ALU control signals (e.g. opcode, func3, func7)
- Design the ALU control unit module
- Write Verilog code for ALU control unit

Step 8: Implement the 7 Instructions

- Implement the R-type instructions (ADD, SUB, AND, OR)
- Implement the I-type instruction (LW)
- Implement the S-type instruction (SW)
- Implement the SB-type instruction (BEQ)

Step 9: Integrate the Components

- Integrate the instruction memory, data memory, program counter, adders, decoder, immediate generation, ALU control unit, and instructions
- Write Verilog code for the integrated processor

Step 10: Write the Testbench

- Write a testbench to simulate and test the processor
- Test the processor with sample instructions and data

Step 11: Simulate and Test

- Simulate the processor using a Verilog simulator
- Test the processor with different instructions and data

Step 12: Debug and Optimize

- Debug any errors or issues found during simulation and testing
- Optimize the processor for performance and area

Step 13: Finalize

- Finalize the Verilog code for the RISC-V single cycle processor

The implementation of the RISC-V processor will be a multi-session project, spanning 3-4 lab sessions.